UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Fecha: 29/03/2026

GUÍA DE EJERCICIOS

PARTE 1 — Dificultad Fácil

1. Representación de un grafo (Lista de Adyacencia)

● Fácil

Dado un número de nodos **n** y una lista de aristas, construya la representación de un grafo utilizando lista de adyacencia. El programa debe mostrar los vecinos de cada nodo.

Implemente la función:

```
func construirGrafo(n: int, aristas: Lista<Par<int,int>>): Lista<Lista<int>>
```

■ *La solución **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m)*

2. Verificación de existencia de arista

● Fácil

Dado un grafo representado mediante **matriz de adyacencia** y dos nodos **u** y **v**, determine si existe una arista directa entre dichos nodos.

```
func existeArista(matriz: Matriz<int>, u: int, v: int): bool
```

■ *La solución **no** debe utilizar apuntadores.* ■ *Complejidad esperada: O(1)*

3. Conteo del grado de un nodo

● Fácil

Dado un grafo no dirigido representado mediante lista de adyacencia, calcule el **grado** de un nodo (cantidad de aristas incidentes).

```
func gradoNodo(grafo: Lista<Lista<int>>, v: int): int
```

■ *La solución **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(grado(v))*

4. Grafo dinámico simple

● Fácil

Implemente una estructura **Grafo** que permita agregar nodos dinámicamente y conectar nodos mediante aristas.

```
func Grafo::agregarNodo(valor: int)
func Grafo::agregarArista(a: int, b: int)
```

✔ *La solución **debe** utilizar apuntadores.* ■ *Complejidad sugerida: O(1).*

PARTE 2 — Dificultad Media

5. Recorrido en Amplitud (BFS)

● Medio

Dado un grafo no dirigido y un vértice inicial, implemente el algoritmo **BFS** mostrando el orden de visita de los nodos.

```
func BFS(grafo: Lista<Lista<int>>, inicio: int): Lista<int>
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

6. Recorrido en Profundidad (DFS)

● Medio

Implemente el algoritmo **DFS** en dos versiones:

- Versión **recursiva**
- Versión **iterativa** utilizando una pila

```
func dfsRecursivo(grafo, inicio)
func dfsIterativo(grafo, inicio)
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

7. Componentes conexas de un grafo

● Medio

Dado un grafo no dirigido, determine cuántas **componentes conexas** existen. Puede apoyarse en BFS o DFS.

```
func componentesConexas(grafo): int
```

■ *La solución **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

8. Red de amistad con nivel de conexión

● Medio

Modele una red social como un grafo no dirigido donde cada nodo representa una persona y cada arista representa una amistad.

```
func RedAmistad::agregarPersona(nombre)
func RedAmistad::agregarAmistad(nombre1, nombre2)
func nivelDeConexion(persona1, persona2)
```

El **nivel de conexión** corresponde al número mínimo de intermediarios entre dos personas (distancia en aristas). Si no existe conexión, retorne -1.

✔ *La implementación **debe** utilizar apuntadores.*

9. Sistema de prerrequisitos de materias

● Medio

Modele el sistema de materias de una universidad como un grafo dirigido, donde una arista **A** → **B** indica que A es prerrequisito de B.

```
func SistemaMaterias::agregarMateria(nombre)
func SistemaMaterias::agregarPrerrequisito(a, b)
func puedeInscribirse(materia, materiasAprobadas)
```

La función **puede_inscribirse** retorna True si el estudiante cumple todos los prerrequisitos directos de la materia objetivo, o False en caso contrario.

✔ *La implementación **debe** utilizar apuntadores.*

10. Detección de ciclos en grafo dirigido

● Medio

Dado un grafo dirigido, determine si existe algún **ciclo** en el grafo utilizando DFS. Considere el estado de cada nodo durante el recorrido.

```
func tieneCiclo(grafo): bool
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

11. Sistema de recomendación de productos

● Medio

Modele un sistema de recomendación como un grafo no dirigido donde los nodos son productos y las aristas indican que dos productos suelen comprarse juntos.

```
func SistemaRecomendacion::agregarProducto(nombre)
func SistemaRecomendacion::agregarRelacion(p1, p2)
func recomendar(producto)
func recomendar_extendido(producto)
```

- **recomendar**: retorna vecinos directos del producto.
- **recomendar_extendido**: retorna productos relacionados hasta 2 niveles de distancia.

✔ *La implementación **debe** utilizar apuntadores.* ■ *Restricción de complejidad: recomendar → O(grado(v)) O(n + m).*

PARTE 3 — Dificultad Difícil

12. Ordenamiento Topológico (Topological Sort)

● Difícil

Dado un grafo dirigido acíclico (**DAG**), implemente el algoritmo de **ordenamiento topológico**. El resultado debe ser una lista de nodos donde cada nodo aparece antes que sus dependientes.

```
func topologicalSort(grafo): Lista<int>
```

■ *La implementación **no** debe utilizar apuntadores.*

13. Camino más corto con pesos (Dijkstra)

● Difícil

Implemente el algoritmo de **Dijkstra** para encontrar las distancias mínimas desde un nodo origen hacia todos los demás nodos del grafo dirigido con pesos positivos. Debe utilizar una **cola de prioridad**.

```
func dijkstra(grafo, origen): Arreglo<float>
```

■ *La implementación **no** debe utilizar apuntadores.*

14. Mapa de rutas entre ciudades

● Difícil

Modele un mapa de ciudades como un grafo no dirigido con pesos, donde cada arista representa una carretera y su peso es la distancia en kilómetros.

```
func agregarCiudad(nombre)
func agregarRuta(ciudad1, ciudad2, distancia)
func ciudadesAMenosDe(ciudad, km)
func esDestinoAlcanzable(origen, destino)
```

- **ciudadesAMenosDe**: retorna todas las ciudades alcanzables con una sola ruta directa de máximo *km* kilómetros.
- **esDestinoAlcanzable**: retorna True si existe algún camino entre ambas ciudades.

✔ *La implementación **debe** utilizar apuntadores.*

15. Puentes en un grafo

● Difícil

Dado un grafo no dirigido, determine todas las **aristas puente**, es decir, aristas cuya eliminación aumenta el número de componentes conexas del grafo.

El algoritmo debe basarse en DFS utilizando **discovery time** y **low values**.

```
func puentes(grafo): Lista<Par<int,int>>
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

16. Puntos de articulación

● Difícil

Dado un grafo no dirigido, determine los **puntos de articulación**, es decir, vértices cuya eliminación desconecta alguna parte del grafo.

La solución debe basarse en DFS con discovery time y low values.

```
func puntosArticulacion(grafo): Lista<int>
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

17. Red eléctrica con detección de fallos

● Difícil

Modele una red eléctrica como un grafo no dirigido donde los nodos son subestaciones y las aristas son líneas de transmisión.

```
func RedElectrica::agregarSubestacion(id)
func RedElectrica::agregarLinea(a, b)
func subestacionesCriticas()
func simularFallo(subestacion)
```

- **subestacionesCriticas**: retorna los puntos de articulación de la red.
- **simularFallo**: elimina temporalmente la subestación y retorna cuántos grupos quedan incomunicados.

✔ *La implementación **debe** utilizar apuntadores.*

18. Componentes fuertemente conexas (Tarjan)

● Difícil

Dado un grafo dirigido, implemente el **Algoritmo de Tarjan** para encontrar todas las componentes fuertemente conexas (SCC). Una SCC es un subconjunto donde cada nodo es alcanzable desde cualquier otro dentro del mismo subconjunto.

```
func tarjan(grafo): Lista<Lista<int>>
```

■ *La implementación **no** debe utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*

19. Planificador de tareas con dependencias y prioridades

● Difícil

Modele un sistema de gestión de proyectos como un **DAG** donde cada nodo es una tarea con nombre, duración estimada en horas y prioridad (1=alta, 2=media, 3=baja). Una arista **A** → **B** indica que A debe completarse antes que B.

```
func agregarTarea(nombre, duracion, prioridad)
func agregarDependencia(a, b)
func ordenEjecucion()
func tiempoMinimoProyecto()
func tareasBloqueadas(tarea)
```

- **ordenEjecucion**: retorna el orden topológico válido respetando todas las dependencias.
- **tiempoMinimoProyecto**: calcula la ruta crítica asumiendo paralelismo donde sea posible.
- **tareasBloqueadas**: retorna todas las tareas que dependen directa o indirectamente de la tarea dada.

✔ *La implementación **debe** utilizar apuntadores.* ■ *Restricción de complejidad: O(n + m).*